

Vibration Monitoring — Technical Report

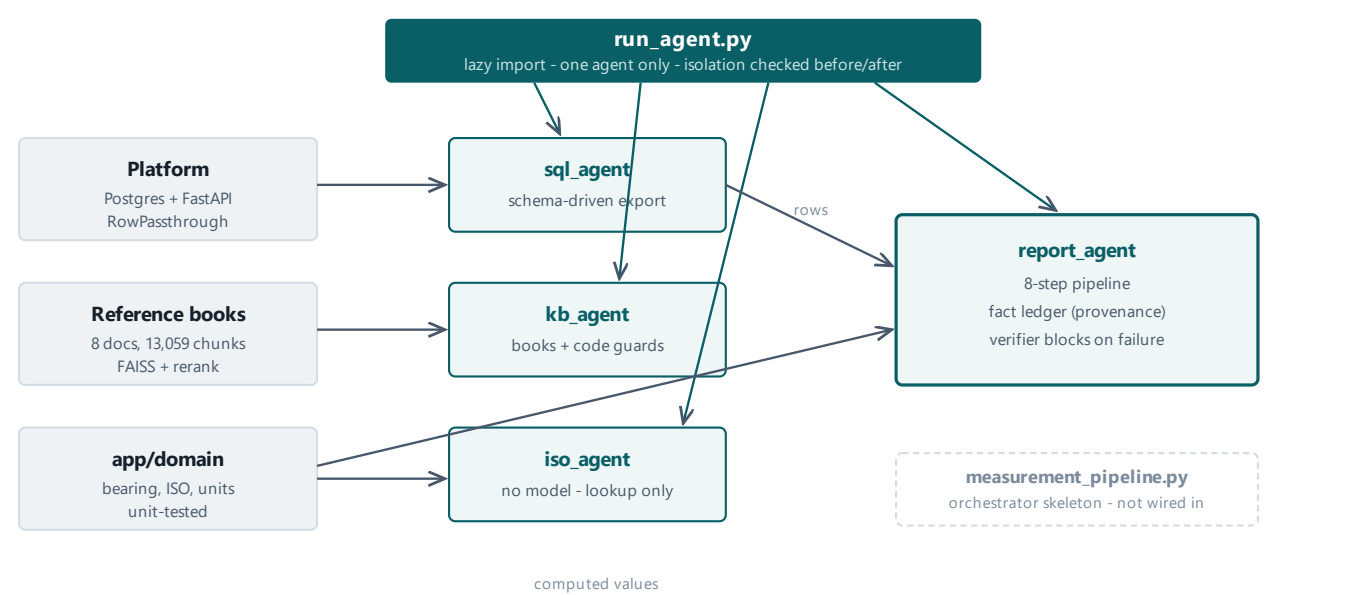
Architecture, calculations, verification work and open items

11 September 2026
4 agents · 260 tests
Target: CWP-001

Scope. Four terminal-only agents and a launcher, built alongside the existing platform and not wired into the dashboard. This document covers what they do, how the calculations work, what was verified and how, what was fixed, and what remains open.

Position. The agents run correctly and are covered by 260 automated tests. The report agent completes all 8 steps, produces 36 provenance-tracked facts and passes its own verifier. What limits its diagnostic value is input data quality, documented in section 6 with the evidence for each claim.

1. Architecture



Data flow. `run_agent.py` loads exactly one agent per invocation. The report agent consumes the SQL agent's rows and the domain library's computed values; the knowledge-base and ISO agents are independent of the report path and of each other.

2. Components

PACKAGE	RESPONSIBILITY	KEY IMPLEMENTATION
<code>sql_agent/</code>	Authenticated HTTP client against the platform API. Flattens sensor, capture and per-channel feature rows into long format and exports them with columns derived from the data received.	<code>dataset.py</code> — <code>_merge()</code> , <code>discover_columns()</code> , <code>schema_report()</code> . <code>CSV_COLUMNS</code> survives as a preferred <i>order</i> , no longer a whitelist (see 7.1)

PACKAGE	RESPONSIBILITY	KEY IMPLEMENTATION
<code>kb_agent/</code>	Question answering over 8 indexed reference documents (13,059 chunks). Per-document FAISS indexes, BGE embeddings, hybrid scoring, cross-encoder rerank.	<code>agent.py</code> , <code>library.py</code> , <code>tools.py</code> , <code>tools_iso.py</code> , <code>verify.py</code>
<code>report_agent/</code>	8-step report pipeline with isolated step failures, provenance ledger, template rendering and post-render verification.	<code>pipeline.py</code> , <code>ledger.py</code> , <code>sources.py</code> , <code>render.py</code> , <code>templates/report.md.j2</code>
<code>iso_agent/</code>	ISO 10816-3 severity limits. Regex question parsing, table lookup, code-assembled answer. No language model takes part , so it needs no API key and loads no index.	<code>agent.py</code> (<code>parse_question</code> , <code>_zone_for</code>), <code>tools.py</code> , <code>__main__.py</code>
<code>run_agent.py</code>	Single entry point. Deferred imports, gpt-4o-mini set centrally, books opt-in behind <code>--with-books</code> , and a before/after isolation check on other agents' write paths.	<code>AgentSpec</code> (<code>writes</code> , <code>depends_on</code>), <code>_fingerprint()</code>
<code>app/domain/</code>	Unit-tested calculation library: bearing geometry and fault frequencies, ISO zone tables, unit conversion.	<code>bearing.py</code> , <code>machine.py</code> , <code>data/bearings.json</code> , <code>data/iso10816_3.json</code>
<code>backend/.../measurement_pipeline.py</code>	Declarative orchestrator skeleton: step table with <code>requires=</code> dependencies, isolated failures, rerun-safety flags. Deliberately not wired into any router.	Demo: <code>backend/scripts/demo_pipeline_orchestrator.py</code>

3. The fact ledger — the central design decision

Measured behaviour drove this. Prompt-level instructions produced probabilistic compliance: a verbatim equation reproduction test passed **2 of 5** runs under instruction alone. Enforced in code the same test passed **5 of 5**.

The generalised rule, and the thing worth carrying into any future agent work: **anything that must be correct has to be enforced in code, not requested in a prompt.**

Mechanism

1. Every quantity is registered in `Ledger` before use, with provenance: **MEASURED** (sensor reading), **COMPUTED** (from the unit-tested domain library, carrying its inputs), **CITED** (indexed document, with book/chapter/page).
2. Templates never write a number. They call `ref(fact_id)`, which emits a placeholder `{{F7}}`. Substitution happens after rendering.
3. `verify()` then checks three things: every placeholder resolved; every referenced fact exists; **no bare number appears in narrative prose**.
4. A verification failure **blocks** the report. It is not a footnote.

A bug worth recording, because it is the failure mode this design invites. The first version ran verification *after* placeholder substitution. At that point every ledger value is a literal digit in the text, so the bare-number check would have passed everything forever while appearing to work. Fixed by keeping both states — `Rendered.raw` (placeholders intact, what the bare-number check reads) and `Rendered.text` (substituted, what the reader gets).

The same class of bug recurred later when an HTML template was trialled: the Markdown-only extractor matched no headings, captured nothing, and passed vacuously. Any verifier must be tested against a known-bad input, not only a known-good one.

3b. Agent separation and the terminal runner

Why the ISO lookup left the knowledge-base agent

It was threaded through `kb_agent` in five places: the tool, a question pattern, a forced lookup that ran in code before the model got a turn, a block that wrote the figures above the prose, and a prompt instruction. All five are gone.

The forced lookup existed because the behaviour was measured. Asked for a 55 kW pump's Zone B/C boundary the model read Table A.2 out of the ingested document and never called the tool — pumps are Groups 3 and 4, which that document does not contain at any power. Handed Group 3 (2.3 / 4.5 / 7.1) and Group 4 (1.4 / 2.8 / 4.5) side by side, it reported 2.8 for both.

The separated agent keeps that property by going further: no model participates at all.

`parse_question()` reads power, machine type, foundation, driver arrangement and any quoted reading with regular expressions; the limits come from `app/domain/iso10816`; the answer is assembled by code. It also grades a reading against the table, and places a value exactly on a boundary in the *lower* zone, per the standard's convention.

Where the machine group is ambiguous — a pump with no driver arrangement stated — it returns every applicable table and names what the question omitted, rather than choosing.

`kb_agent` no longer exposes the ISO tool to the model, and `_refer_limits_to_iso_agent()` appends a pointer to any limits question, so a reader who asks for a number is told where the number lives instead of being handed prose they might mine a figure from.

A defect the new tests surfaced, present in the original pattern and fixed in both copies: the limit-question regex required the word "zone" twice, so "zone B/C boundary" — the usual phrasing — was never recognised.

The runner, and isolation that is checked rather than claimed

```
python run_agent.py list
python run_agent.py iso      "zone B/C for a 55 kW pump, separate driver, rigid"
python run_agent.py sql      data "cooling water pump"
python run_agent.py report build "cooling water pump" --separate-driver
python run_agent.py kb --with-books ask "what causes oil whirl"
```

1. **Deferred imports.** `importlib.import_module` runs after the agent is chosen, so the SQL agent pulls in no faiss, no embedding model and no book index. Measured: 0.62 s, reporting *"this agent only - nothing else was imported"*.
2. **Books opt-in.** `kb` refuses to start without `--with-books`, being the only agent that loads the index and calls a paid API.
3. **One model.** `OPENAI_MODEL` is set to `gpt-4o-mini` before any agent imports, so a run cannot silently cost more than the last.
4. **Isolation check.** Each agent declares the paths it may write. Every *other* agent's paths are fingerprinted before the run (file count, total bytes, newest mtime) and compared after.
5. **Declared dependencies vs leaks.** `report_agent` legitimately builds on `sql_agent`; that reads as `depends on: sql_agent` (declared), while anything undeclared is reported as unexpected.

The check was verified by forcing a real violation, not by inspection: pointing the report agent's `--out` into the SQL agent's export directory produced

```
ISOLATION : FAILED - this run modified sql:data/exports
```

A check that cannot fail proves nothing — the same lesson as the verifier bug in section 3.

4. RPM calculation — specification

Implemented in `backend/app/services/feature_extraction.py`. Consumed and reconciled in `report_agent/sources.py::derive_shaft_speed()`.

Backend, per channel

```
SHAFT_FREQ_MIN_HZ = 5.0
SHAFT_FREQ_MAX_HZ = 120.0

w      = np.hanning(n)                # taper, kills spectral leakage
spec   = abs(fft(x * w))[:n//2] * (2.0 / w.sum()) # normalise by coherent gain
freqs  = fftfreq(n, d=1.0/fs)[:n//2]

mask   = (freqs >= 5.0) & (freqs <= 120.0)      # 300-7200 rpm
shaft_hz = freqs[mask][argmax(spec[mask])]      # tallest peak in band

# stored into the metadata of amplitude_1x / 2x / 3x
shaft_meta = {"estimated_shaft_hz": shaft_hz, "sampling_rate_hz": fs, "sample_count": n}
```

Report agent, across channels

Reads `estimated_shaft_hz` per channel, tallies values, takes the modal value as consensus, records dissenters, sets `confidence = len(agreeing) / len(per_channel)`, and emits `round(consensus_hz * 60)` rpm as a COMPUTED fact.

5. RPM verification — what was actually checked

Three independent checks against `backend/sample_vibration_data.csv` (8192 samples, 8 channels), not a re-reading of the code.

Check A — does the stored value match what the algorithm produces?

The backend algorithm was reimplemented from scratch and run over the raw file. It reproduces **25.0 Hz on 7 channels and 50.0 Hz on channel 3**, matching the database exactly. No transport or storage fault.

Check B — is the answer physically right?

Autocorrelation was used as a wholly independent method (no FFT, no peak picking).

CHANNEL	BACKEND (FFT)	AUTOCORRELATION	RATIO	READING
0, 1, 2, 6	25.04 Hz	~25.1 Hz	1.00	agree
3	50.08 Hz	25.11 Hz	1.99	backend took 2X
4	25.04 Hz	8.17 Hz	3.07	disagreed
5	25.04 Hz	12.56 Hz	1.99	disagreed
7	25.04 Hz	63.31 Hz	0.40	disagreed

Check C — which method was fooled?

Autocorrelation has a weakness that mirrors the FFT's: a signal repeating every cycle also repeats every 2nd and 3rd cycle, so it can lock onto a *sub*-harmonic exactly as peak-picking locks onto a harmonic. Direct inspection of the spectra settles it:

CHANNEL	8.2 HZ	12.6 HZ	25 HZ	50 HZ	DOMINANT
3	0.0002	0.0004	0.1796	0.5506	50.1 Hz
4	0.0003	0.0003	0.0587	0.0245	25.0 Hz
5	0.0596	0.1204	0.3008	0.2196	25.0 Hz
7	0.0083	0.0084	0.0392	0.0040	25.0 Hz

Verdict. The backend is **correct on channels 4, 5 and 7** — the 25 Hz component is genuinely dominant there and my autocorrelation check was the one fooled, by sub-harmonics. Channel 3 is a **confirmed 2X harmonic lock**: its 50 Hz peak measures 0.5506 against 0.1796 at 25 Hz, roughly three times taller, so "tallest peak in band" legitimately selects the wrong component.

The algorithm and its stored output are sound. The single real weakness is harmonic lock, and the cross-channel vote already contains it.

Defect found: stored sampling rate does not match the file

SOURCE	SAMPLING RATE	SHAFT SPEED	RESULT
File's own timestamps (median Δt = 39.0 μs)	25,641 Hz	25.040 Hz	1502.4 rpm
Recorded in <code>sensor_baselines.sampling_rate_hz</code>	25,600 Hz	25.000 Hz	1500.0 rpm

A **0.16% discrepancy**. The stored rate is a configured value, not derived from the capture. Immaterial at this magnitude — both land within 1.5% of the 1480 rpm nameplate — but it is a real inconsistency and the mechanism

by which a larger error could enter unnoticed. The whole frequency axis scales linearly with this number.

Resolution — and a correction to how the nameplate agreement should be read

The binning is the dominant limit on this method and is easy to overlook:

```
Δf = fs / n = 25600 / 8192 = 3.125 Hz    -> 3.125 x 60 = 187.5 rpm per bin

bin 7 = 21.875 Hz = 1312.5 rpm
bin 8 = 25.000 Hz = 1500.0 rpm  <- selected
bin 9 = 28.125 Hz = 1687.5 rpm
```

The estimate is quantised to multiples of 187.5 rpm. The nameplate 1480 rpm is 24.667 Hz, which falls between bins: **the FFT cannot report 1480 at all**, and 1500 is the nearest value it is capable of producing.

So the earlier framing of this as "1.35% apart, independent corroboration" overstated it and is corrected here. The agreement establishes that the correct *bin* was selected — not a 2X or 3X harmonic, not a noise peak. That is the check worth having and it passes. It is not evidence of 1.35% accuracy, because no closer answer was available to the method.

Parabolic interpolation over the winning bin and its neighbours gives a sub-bin estimate and shows the true peak is centred on bin 8 rather than pulled toward 1480:

```
δ = 0.5 * (A[k-1] - A[k+1]) / (A[k-1] - 2*A[k] + A[k+1])
shaft_hz = (k + δ) * Δf

bins 7,8,9 = 0.02243, 0.04489, 0.02250    -> δ = +0.0007 bins
refined    = 25.0023 Hz = 1500.1 rpm
```

Consistent with the capture having been generated at exactly 25 Hz, which is itself further evidence the data is synthetic (finding 5).

Recommended changes

1. **Harmonic check before the vote.** Where a candidate is an integer multiple of another candidate inside the band, prefer the lower unless the higher is dominant by a wide margin. This is the channel-3 case and the only demonstrated weakness.
2. **Derive the sampling rate from capture timestamps**, or reconcile the configured value against them and flag a mismatch beyond a tolerance.
3. **Automatic nameplate sanity check.** `rated_rpm` is already in the database and joinable; flag an estimate more than a few percent away.
4. **Populate `rotation_speed_rpm` at capture time** where a tachometer exists. It removes the estimate and its uncertainty entirely.
5. **Confirm the 5–120 Hz band** (300–7200 rpm) covers the intended fleet.
6. **Parabolic peak interpolation.** Three lines, and it lifts the estimate off the 187.5 rpm grid to a sub-bin figure. Cheapest accuracy improvement available.
7. **Longer captures.** $\Delta f = f_s/n$, so resolution is set by capture *duration*, not sampling rate. 0.32 s gives 187.5 rpm steps; 1.28 s would give 47 rpm steps.

6. Data findings, with evidence

#	FINDING	EVIDENCE	STATUS
1	Shaft speed absent from every capture; recoverable from feature metadata	<code>rotation_speed_rpm</code> null on 9/9; <code>estimated_shaft_hz</code> present on all channels	FIXED
2	No conversion path from stored units to ISO velocity	<code>feature_extraction.py</code> passes CSV values through as <code>scaled_eng</code> ; sensitivity read only by <code>acquisition_config.py</code>	FIXED
3	No usable baseline — the one on file is the same data as the measurements	<code>sensor_baselines</code> : 1 row, <code>is_primary=f</code> , 0 rows in <code>baseline_channel_features</code> ; 144 <code>no_baseline</code> = 2 features × 8 ch × 9 captures	BLOCKED
4	Thresholds cannot be scoped per sensor	<code>feature_threshold_rules</code> has no <code>sensor_id</code> column; 0 machine-type rules, 0 channel rules; RMS 72/72 critical, range 0.024–0.424 vs limit 0.02	BLOCKED
5	All 9 captures are numerically identical	MD5 over each upload's ordered feature vector → 9 uploads, 1 distinct fingerprint ; holds across both filenames	BLOCKED
6	Machine profile exists but is not consumed by the report agent	<code>sensor_configurations.equipment_id</code> joins to CWP-001: 75 kW, 1480 rpm, DE 6312-C3, NDE 6310-C3, concrete foundation	NEXT

Threshold scoping — what is available versus what is honoured

SCOPE	SCHEMA	HONOURED AT FEATURE-COMPUTE TIME	RULES SET
Per channel	<code>channel</code> column	Yes — <code>resolve_rule()</code> , override with global fallback	0
Per machine type	<code>machine_type</code> column	No — <code>persist_upload_features_and_trends()</code> calls <code>get_resolved_rule_map(db)</code> with no machine type, so typed rules are ignored	0
Per sensor	no column	Not possible	—

Rule types and baseline dependency

Only `percent_baseline` returns `STATUS_NO_BASELINE` . Two of ten features use it, which accounts for all 144 unassessed readings; the remaining 576 assessments require no baseline.

RULE TYPE	FEATURES	NEEDS BASELINE
<code>absolute_max</code>	rms, peak, kurtosis	No
<code>absolute_db</code>	noise_floor	No
<code>range</code>	crest_factor	No

RULE TYPE	FEATURES	NEEDS BASELINE
percent_rms	amplitude_1x / 2x / 3x	No — self-referencing
percent_baseline	envelope_rms, fft_band_energy_0_500	Yes

7. Fixes applied

7.1 Metadata passthrough — shaft speed recovery

`sql_agent/dataset.py` built rows from a fixed `CSV_COLUMNS` list of ~25 names. `metadata` was absent, so `estimated_shaft_hz` was discarded in transit. Adding the passthrough leaves the exported CSV byte-identical, because the writer is constructed with `extrasaction="ignore"`.

Result: **1500 rpm**, 7-of-8 consensus, confidence 0.875, corroborated by nameplate.

This is the concrete argument for making the SQL agent schema-driven: a hardcoded column list silently drops anything the platform later adds.

7.2 Velocity conversion

`convert_to_velocity()` integrates 1X acceleration at the derived shaft speed. It refuses to infer the input unit, and the refusal is defensible from the source rather than merely cautious: the feature path never applies sensitivity, so the stored value's physical meaning is unrecorded. As g: 34.4 mm/s. As volts: 343.7 mm/s. ISO damage boundary: 7.1–11.0 mm/s. Both implausible, so no zone is assigned and the report carries an explicit warning.

8. Open items

#	WORK	NATURE	EFFORT
1	Make <code>sql_agent</code> schema-driven — done , 27 to 97 columns; <code>RowPassthrough</code> opened the API layer and <code>schema_drift.py</code> reports anything the models do not declare	Code	DONE
2	Extract the ISO tool from <code>kb_agent</code> into a standalone agent — done , see section 3b	Code	DONE
3	Apply the RPM recommendations in section 5	Backend	Small
4	Terminal runner with per-agent isolation — done , <code>run_agent.py</code> , see section 3b	Code	DONE
5	Join the report agent to <code>equipment_masters</code> — removes three CLI flags, unlocks bearing frequencies	Code	Small
6	Per-sensor threshold scoping	Migration + UI	Large
7	Real captures and a healthy baseline	Plant, not code	—

What item 5 would unlock, computed from the live record at 1500 rpm

BEARING	BPFO	BPFI	BSF	FTF
6312-C3 (DE)	87.6 Hz (3.51X)	137.4 Hz (5.49X)	53.8 Hz	9.74 Hz
6310-C3 (NDE)	77.5 Hz (3.10X)	122.5 Hz (4.90X)	52.7 Hz	9.69 Hz

Geometry estimated from ISO 15 boundary dimensions, confidence 0.6 — the catalogue holds internal geometry for 6203 and 6205 only. Ball count may be off by one, shifting BPFO/BPFI ~ 10%. Adding real geometry for 6312/6310 to `app/domain/data/bearings.json` raises this to 1.0.

9. Running the agents

```
# from vibrationbot/ , with the platform containers running

python run_agent.py list                # what is available
python run_agent.py iso    "zone B/C for a 55 kW pump, separate driver, rigid"
python run_agent.py sql    data "cooling water pump"
python run_agent.py report build "cooling water pump" --separate-driver --acceleration-unit g
python run_agent.py kb --with-books ask "what causes oil whirl"

# each agent still runs directly if preferred:
python -m report_agent build "cooling water pump" --separate-driver --acceleration-unit g
python -m report_agent build "cooling water pump" --out report.md
python -m report_agent build --help

# every run ends with:
#   steps      : collect=ok, integrity=ok, features=ok, iso=ok, shaft=ok,
#               velocity=ok, calibration=ok, assess=ok
#   facts      : 36
#   verified   : PASS - every figure traces to the ledger
```

Terminal output is transliterated to ASCII (cp1252 consoles cannot encode the report's punctuation); `--out` preserves UTF-8. Step failures are isolated — a failed step marks its section incomplete with a stated reason rather than aborting the report.

10. Test coverage

260 test functions. Full suite passes. Coverage concentrates on the domain library (bearing frequencies and ISO zones against published references), the ledger and verifier, ingestion and section-path repair, the retrieval path, the schema-driven export, and the ISO agent's separation — the last verified in a fresh interpreter, because this session has already imported `kb_agent` and would otherwise pass for the wrong reason.

The knowledge-base agent additionally has a 23-question evaluation harness across six answer classes (`answerable` , `partial` , `absent` , `figure_only` , `verbatim` , `scoped_value`) with five automated checks including citation-topic mismatch — built because correct-looking prose was passing manual review while citing the wrong source.

Vibration monitoring — technical report · 11 September 2026.

Every figure was read from the running database, computed by the unit-tested domain library, or produced by the verification scripts described in section 5, at the time of writing. A companion plain-language document covers the same ground without technical detail.